AGATE Tosca Migration – User Manual

- 1. Einleitung
- 2. Voraussetzungen
- 3. Tosca Export vorbereiten
  - 3.1 Template exportieren
  - 3.2 TestSheet / TestCase Design exportieren
- 4. Grundsyntax des Migrators
- 5. Cleanup-Modi
  - 5.1 Cleanup Dry Run
    - Empfehlung
  - 5.2 Cleanup
- 6. Template migrieren
- 7. TestCase Design migrieren
- 8. Besonderheit bei Template- und TCD-Dateinamen
- 9. Generierte Dateistruktur
- 10. Beispiel: Migrierter AGATE Test
- 11. REST- und SOAP-Module
- 12. metadata.json
- 13. Customer-spezifische Tosca-Übersetzungen
- 14. Buffer Mappings
- 15. Setting Mappings
- 16. Literal Mappings
- 17. Unterstützte Tosca Standardmodule
- 18. Beispiel: TBox Set Buffer
- 19. Reusable TestStepBlocks
- 20. Control Flow
- 21. Speziell behandelte Tosca Module
- 22. Tosca Migration Review
- 23. Migration File Audit
- 24. Migration Manifeste
- 25. Separate clean-Funktion
- 26. Manuelle Nachkontrolle
- 27. Leere Root-YAML-Dateien
- 28. Vollständiges Beispiel
  - Schritt 1 – Tosca Export
  - Schritt 2 – Template zunächst als Dry Run migrieren
  - Schritt 3 – Template mit Cleanup migrieren
  - Schritt 4 – TestCase Design als Dry Run migrieren
  - Schritt 5 – TestCase Design migrieren
  - Schritt 6 – Ergebnis prüfen
  - Schritt 7 – Migration Review prüfen
  - Schritt 8 – Nicht benötigte Dateien entfernen
- 29. Übergabe an den AGATE Server
- 30. Empfohlener Workflow
- 31. Quick Reference
  - Template Migration
  - TestCase Design Migration
  - Testcase Cleanup
  - Template Cleanup
  - Translation Config überschreiben
  - Migration-Ergebnis

Testausführung
32. Zusammenfassung

1. Einleitung

Der **AGATE Tosca Migrator** dient zur automatisierten Überführung bestehender Tricentis-Tosca-Testfälle in das AGATE-Testformat.

Der Migrator analysiert exportierte Tosca-TSU-Dateien und erzeugt daraus – abhängig vom Inhalt des Tosca-Tests – unter anderem:

- AGATE Test-Suites im YAML-Format
- AGATE Templates
- CSV-Dateien für TestCase-Design-Daten
- Reusable Testbausteine
- REST- und SOAP-Module
- SQL-, BUFFER-, WAIT-, FILE- und weitere AGATE-Schritte
- Migrations- und Cleanup-Informationen

Der Migrator ist ausschließlich für die **Migration** zuständig.

Er führt die erzeugten AGATE-Tests **nicht selbst aus**.

Die generierten Dateien unter

```
migration/data/<appID>/

```

können nach der Migration und einer fachlichen Prüfung in die entsprechende Datenstruktur des **agate-server** übernommen werden.

Die eigentliche Testausführung erfolgt anschließend über den AGATE Server bzw. den normalen AGATE Test Runner.

Das grundlegende Prinzip lautet daher:

```
Tosca
   |
   v
AGATE Tosca Migrator
   |
   v
migration/data/<appID>/
   |
   v
fachliche Prüfung / ggf. Korrektur
   |
   v
agate-server
   |
   v
Testausführung

```

---

2. Voraussetzungen

Für die Migration werden benötigt:

- ein gebauter AGATE Tosca Migration JAR
- Java im `PATH`
- ein oder mehrere Tosca TSU-Exporte
- eine Tosca-Translation-Konfiguration
- ein Application-Identifier (`appID`)

Der Migration JAR befindet sich standardmäßig unter:

```
target/agate-tosca-migration-svc-1.0.0-SNAPSHOT-jar-with-
dependencies.jar
```

Falls der JAR noch nicht erzeugt wurde:

```
mvn clean package
```

Die Tosca TSU-Dateien müssen unter:

```
tsu/
```

liegen.

Beispiel:

```
tsu/
├── T_AGATE_Demo.tsu
└── TCD_AGATE_Demo.tsu
```

---

3. Tosca Export vorbereiten

Bei einem Tosca Template mit TestCase Design werden in der Regel zwei Exporte benötigt.

3.1 Template exportieren

Der erste Export enthält das Tosca Template bzw. den eigentlichen Testfall.

Beispiel:

```
T_AGATE_Demo.tsu
```

Dieser Export enthält unter anderem:

- TestSteps
- verwendete Tosca Module
- Buffer-Operationen
- API-Aufrufe
- Reusable-Referenzen

- Testlogik

3.2 TestSheet / TestCase Design exportieren

Wenn der Tosca-Test TestCase Design verwendet, wird zusätzlich das entsprechende TestSheet exportiert.

Beispiel:

```
TCD_AGATE_Demo.tsu
```

Dieser Export enthält die TestCase-Design-Struktur und die zugehörigen Instanzwerte.

---

4. Grundsyntax des Migrators

Eine Migration wird über folgende Syntax gestartet:

```
startToscaMigration.bat migrate <appID> <baseFileName>
```

Beispiel:

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo"
```

Dabei bedeutet:

Parameter	Beschreibung
`migrate`	Startet eine Tosca-zu-AGATE-Migration
`<appID>`	Ziel-Applikation innerhalb der AGATE-Datenstruktur
`<baseFileName>`	Name der TSU-Datei ohne `.tsu`

Aus:

```
tsu/T_AGATE_Demo.tsu
```

wird somit:

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo"
```

---

5. Cleanup-Modi

Der Migrator unterstützt zusätzlich eine automatische Analyse der während der Migration erzeugten Dateien.

Dafür stehen zwei Modi zur Verfügung:

```
--cleanup-dry-run
--cleanup

```

5.1 Cleanup Dry Run

Empfohlen wird, eine neue oder noch nicht geprüfte Migration zunächst mit:

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo" --cleanup-
dry-run

```

auszuführen.

Der Dry Run führt die vollständige Migration und anschließend den Migration File Audit aus.

Dabei wird analysiert:

- welche Dateien neu erzeugt wurden
- welche bestehenden Dateien verändert wurden
- welche Dateien vom aktuellen Test tatsächlich benötigt werden
- welche Dateien über frühere Migration-Manifeste geschützt sind
- welche Dateien potentielle Cleanup-Kandidaten sind

Der Dry Run **löscht keine Dateien**.

Im Log erscheint beispielsweise:

```
CLEANUP DRY-RUN:

  CANDIDATE -> modules/rest/example/request.json

```

Damit kann vor dem tatsächlichen Löschen geprüft werden, ob die Dependency-Erkennung korrekt funktioniert.

Empfehlung

Bei neuen oder komplexen Migrationen sollte zunächst immer:

```
--cleanup-dry-run

```

verwendet werden.

Erst nachdem die angezeigten Cleanup-Kandidaten geprüft wurden, sollte ein Lauf mit:

```
--cleanup

```

erfolgen.

5.2 Cleanup

Mit:

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo" --cleanup

```

wird nach der Migration ebenfalls ein Artifact Audit durchgeführt.

Im Unterschied zum Dry Run werden zulässige Cleanup-Kandidaten anschließend tatsächlich gelöscht.

Beispiel:

```
CLEANUP PLAN:

  DELETE -> modules/rest/old_module/metadata.json
  DELETE -> modules/rest/old_module/request.json

```

Der Cleanup arbeitet bewusst restriktiv.

Geschützt werden insbesondere:

- bereits bestehende Dateien
- aktuell verwendete Dateien
- Root- und Template-Dateien
- zugehörige CSV-Dateien
- Migration-Manifeste
- Dateien, die über frühere Migrationen noch benötigt werden

Das Ziel ist, veraltete neu erzeugte Migrationsartefakte zu entfernen, ohne bereits bestehende oder noch verwendete AGATE-Artefakte unbeabsichtigt zu löschen.

---

6. Template migrieren

Für unser Beispiel:

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo" --cleanup

```

Der Migrator verarbeitet den Tosca-Export und erzeugt die entsprechenden AGATE-Artefakte.

Typische Zwischenverarbeitung umfasst:

```
TSU Extraction
      |
      v
Extended JSON
      |
      v
Compressed JSON
      |
      v
```

```
Parser Phase
      |
      v
IF / Folder Propagation
      |
      v
TCD Processing
      |
      v
AGATE YAML / Module / CSV Generation
      |
      v
Migration Review
      |
      v
Artifact Audit / Cleanup

```

Die internen JSON-Dateien unter `jsonOut` dienen dabei hauptsächlich der Transformationspipeline und sind normalerweise nicht das eigentliche Ergebnis, das später im AGATE Server benötigt wird.

---

7. TestCase Design migrieren

Für Tosca TestCase Design wird zusätzlich der TestSheet-Export migriert.

Beispiel:

```
startToscaMigration.bat migrate TOSCATOAGATE "TCD_AGATE_Demo" --cleanup

```

Der TCD-Migrationsprozess analysiert:

```
TestSheet
   |
   v
Attribute / Relation Struktur
   |
   v
TestCase Design Instances
   |
   v
hierarchische Parameterwerte
   |
   v
flache Testdaten
   |
   v
CSV

```

Das Ergebnis wird unter:

```
migration/data/<appID>/template/

```

gespeichert.

Beispiel:

```
migration/data/TOSCATOAGATE/template/TCD_AGATE_Demo.csv

```

Die CSV-Datei enthält die aus Tosca TestCase Design übernommenen Testdaten.

---

8. Besonderheit bei Template- und TCD-Dateinamen

Tosca Template und Tosca TestSheet müssen nicht zwingend exakt gleich benannt sein.

Aus diesem Grund kann die Template-Migration unter Umständen zusätzlich eine CSV-Datei erzeugen, deren Name nicht dem später tatsächlich benötigten TestSheet entspricht.

Beispiel:

```
template/T_AGATE_Demo.csv
template/TCD_AGATE_Demo.csv

```

Im Beispiel gehört der eigentliche TestCase-Design-Export zu:

```
TCD_AGATE_Demo

```

Nach der separaten TCD-Migration ist daher zu prüfen, welche CSV-Datei tatsächlich zum migrierten AGATE Template gehört.

Nicht benötigte oder doppelt erzeugte CSV-Dateien können anschließend entfernt werden.

Diese Prüfung ist insbesondere dann wichtig, wenn Template- und TestSheet-Namen in Tosca nicht nach einem einheitlichen Namensschema aufgebaut sind.

---

9. Generierte Dateistruktur

Nach einer vollständigen Migration kann die Struktur beispielsweise so aussehen:

```
migration/
└── data/
    └── TOSCATOAGATE/
        ├── .migration-manifests/
        ├── modules/
        │   └── rest/
        │       └── v4/
        │           └── math_request/
        │               ├── metadata.json
        │               └── request.json
        ├── reusable/
        │   └── ...
        └── template/
            ├── AGATE_Demo.yaml
            └── TCD_AGATE_Demo.csv

```

Nicht jedes Projekt benötigt alle Verzeichnisse.

Beispielsweise existiert `reusable/` nur dann, wenn tatsächlich Tosca Reusable-Blöcke migriert werden.

---

10. Beispiel: Migrierter AGATE Test

Der Tosca-Test `AGATE_Demo` enthält drei Buffer-Operationen und einen REST-Aufruf.

Das Ergebnis ist beispielsweise:

```
- id: "AGATE_Demo"
  description: "AGATE_Demo"
  stage: "*"
  priority: HIGH

  variables:
    {}

  steps:

    - type: BUFFER
      op: EXEC
      name: 'x'
      value: '{XL[x]}'

    - type: BUFFER
      op: EXEC
      name: 'y'
      value: '{XL[y]}'

    - type: BUFFER
      op: EXEC
      name: 'z'
      value: '{XL[z]}'

    # Math-Request
    - type: REST
      op: EXEC
      endpoint: "{E[env.math.url]}"
      command: rest.v4.math_request
      response: math_request_response

```

Die Werte:

```
{XL[x]}
{XL[y]}
{XL[z]}

```

werden aus der mit dem Template verbundenen CSV-Datei gelesen.

Damit können mehrere Tosca TestCase-Design-Instanzen mit demselben AGATE Template ausgeführt werden.

---

11. REST- und SOAP-Module

Tosca API Module werden nicht vollständig in den eigentlichen Testschritt eingebettet.

Stattdessen erzeugt der Migrator separate AGATE API-Module.

Für:

```
command: rest.v4.math_request

```

wird beispielsweise folgende Struktur erzeugt:

```
modules/
```

```
└── rest/
    └── v4/
        └── math_request/
            ├── metadata.json
            └── request.json

```

Der YAML-Test verweist lediglich über:

```
command: rest.v4.math_request

```

auf dieses Modul.

Der Modulname und der physische Modulpfad müssen dabei zusammenpassen:

```
rest.v4.math_request

        ↓

modules/rest/v4/math_request/

```

---

12. metadata.json

Die `metadata.json` beschreibt den technischen API-Aufruf.

Im Beispiel:

```
{
  "url" : "{{endpoint}}/v4/?expr={B[x]}*({B[y]}-{B[z]})",
  "method" : "GET",
  "headers" : {
    "Content-Type" : "application/json;charset=UTF-8"
  }
}

```

Der Endpoint selbst wird vom Test geliefert:

```
endpoint: "{E[env.math.url]}"

```

Dadurch bleibt die technische Request-Definition vom konkreten Environment getrennt.

---

13. Customer-spezifische Tosca-Übersetzungen

Nicht alle Tosca Variablen sollen unverändert nach AGATE übernommen werden.

Dafür verwendet der Migrator eine Translation-Konfiguration.

Standardmäßig:

```
agate-acme-config/acme-tosca-translations.yaml

```

Die Konfiguration kann über eine Environment Variable überschrieben werden:

```
set AGATE_TOSCA_TRANSLATION_CONFIG=C:\path\customer.yaml

```

Damit können unterschiedliche Kunden bzw. Projekte eigene Mapping-Regeln verwenden.

---

14. Buffer Mappings

Beispiel aus Tosca:

```
{B[MATH_URL]}
```

Die Translation-Konfiguration enthält:

```
bufferMappings:

  MATH_URL: env.math.url
```

Der Migrator erzeugt daraus:

```
{E[env.math.url]}
```

und im fertigen AGATE Test:

```
endpoint: "{E[env.math.url]}"
```

Weitere Beispiele:

```
bufferMappings:

  ACME_DB_USER: env.database.user
  ACME_DB_PASSWORD: env.database.password

  ACME_OpenShift_Namespace: env.openShift.namespace
  G_ACME_OpenShift_Namespace: env.openShift.namespace

  MATH_URL: env.math.url
```

---

15. Setting Mappings

Auch Tosca Settings können auf AGATE Environment-Werte abgebildet werden.

Beispiel:

```
{S[SVC.Projekt root path]}
```

wird mit:

```
settingMappings:

  "SVC.Projekt root path": env.SVC_Projekt_root_path
```

zu:

```
{E[env.SVC_Projekt_root_path]}
```

---

16. Literal Mappings

Zusätzlich können projektspezifische Textwerte ersetzt werden.

Beispiel:

```
literalMappings:

  '%logcheckcloud%\%LogCheckCloud%': '{E[env.openshift_logcheck_dir]}'
  '%logcheckcloud%': '{E[env.openshift_logcheck_dir]}'

```

Damit können technische Pfade oder projektspezifische Tosca-Konstanten bereits während der Migration in das gewünschte AGATE-Format überführt werden.

---

17. Unterstützte Tosca Standardmodule

Der Migrator unterstützt bereits verschiedene Tosca Standardmodule.

Aktuell gehören dazu unter anderem:

Tosca Modul / Konzept	AGATE Ziel	Beschreibung
TBox Set Buffer	`BUFFER`	Tosca Buffer-Werte werden in AGATE Buffer-Schritte überführt.
TBox DB Expert module	`SQL`	Datenbankabfragen und SQL-Operationen werden in AGATE SQL-Schritte transformiert.
TBox DB Run SQL Statement	`SQL`	SQL Statements werden in AGATE SQL-Schritte übernommen.
TBox Wait	`WAIT`	Tosca Wait-Schritte werden als AGATE WAIT-Schritte erzeugt.
TBox Start Program	`CMD`	Start externer Programme bzw. Commands.

TBox Read/Create File	`FILE`	Tosca Dateioperationen werden auf die AGATE File Engine abgebildet.
TBox Evaluation Tool	`ASSERT` / Control Flow	Evaluation-Ausdrücke werden abhängig vom Kontext als Assertion oder Bestandteil der Control-Flow-Logik verarbeitet.
Reusable TestStepBlock	`CALL`	Tosca Reusable-Blöcke werden als AGATE Reusable YAML erzeugt und über CALL verwendet.
ApiModule	`REST` / `SOAP`	Tosca API Module werden in AGATE REST- bzw. SOAP-Module überführt.

Die Liste wird mit der Weiterentwicklung des Migrators erweitert.

---

18. Beispiel: TBox Set Buffer

Ein Tosca:

```
TBox Set Buffer

```

für den Buffer:

```
x

```

kann beispielsweise erzeugen:

```
- type: BUFFER
  op: EXEC
  name: 'x'
  value: '{XL[x]}'

```

Dadurch wird die Tosca Buffer-Logik in die AGATE Buffer Engine übertragen.

---

19. Reusable TestStepBlocks

Tosca Reusable TestStepBlocks werden nach:

```
migration/data/<appID>/reusable/

```

migriert.

Ein Aufruf eines solchen Blocks wird im AGATE-Test anschließend über einen `CALL`-Schritt abgebildet.

Das Ziel ist, die Wiederverwendung bestehender Tosca-Testlogik auch nach der Migration beizubehalten.

---

20. Control Flow

Neben normalen Tosca Modulen verarbeitet der Migrator auch bestimmte Steuerungsstrukturen.

Dazu gehören insbesondere:

```
IF
LOOP
BREAK

```

Diese werden während der Transformationspipeline in native AGATE-Control-Flow-Strukturen überführt.

Dadurch muss Tosca-Control-Flow nicht durch technische Hilfskonstrukte im fertigen AGATE YAML nachgebildet werden.

---

21. Speziell behandelte Tosca Module

Nicht jedes Tosca-Modul benötigt eine direkte 1:1-Entsprechung in AGATE.

Ein Beispiel ist:

```
TBox Delete Buffer

```

AGATE verwaltet den Lebenszyklus der Buffer selbst.

Deshalb ist nicht zwingend ein entsprechender expliziter AGATE-Schritt erforderlich.

Andere Module, für die der Migrator noch keine automatische Transformation besitzt, sollen nicht stillschweigend verschwinden.

Sie werden über den Migration Review ausgewiesen.

---

22. Tosca Migration Review

Am Ende der Migration erzeugt der Migrator einen Review.

Bei einem vollständig verarbeiteten Test kann beispielsweise erscheinen:

```
============================================================
TOSCA MIGRATION REVIEW
============================================================
[SUCCESS] No unprocessed Tosca modules found.
============================================================

```

Wenn ein Tosca-Modul noch nicht unterstützt wird, wird es dort ausgewiesen.

Der Migration Review dient daher als Checkliste für notwendige manuelle Nacharbeiten.

Ein erfolgreicher Migration-Lauf bedeutet:

```
Migration technisch abgeschlossen

```

aber nicht automatisch:

```
Test fachlich vollständig verifiziert

```

Der migrierte Test sollte deshalb vor der Übernahme in den AGATE Server geprüft werden.

---

23. Migration File Audit

Wenn ein Cleanup-Modus aktiviert wurde, analysiert der Migrator die erzeugten Dateien.

Der Audit zeigt beispielsweise:

```
Files before run
Files after run
NEW
EXISTING
MODIFIED
REACHABLE CURRENT
CLEANUP CANDIDATES

```

Die wichtigsten Bedeutungen sind:

Status	Bedeutung
`NEW`	Datei wurde in dieser Migration neu erzeugt
`EXISTING`	Datei existierte bereits
`MODIFIED`	Bestehende Datei wurde verändert
`REACHABLE CURRENT`	Datei wird vom aktuell migrierten Test benötigt

`UNUSED/CANDIDATE`	Neue Datei wird aktuell nicht referenziert und kann ein Cleanup-Kandidat sein
`PROTECTED`	Datei wird durch aktuelle oder frühere Migrationen geschützt

---

24. Migration Manifeste

Der Migrator erzeugt zusätzlich Manifest-Dateien unter:

```
migration/data/<appID>/.migration-manifests/
```

Beispiel:

```
.migration-manifests/T_AGATE_Demo.json
```

Diese Informationen helfen dem Cleanup zu erkennen, welche Dateien durch aktuelle oder frühere Migrationen benötigt werden.

Dadurch soll verhindert werden, dass ein späterer Migrationslauf eine Datei entfernt, die von einer bereits migrierten Test-Suite noch benötigt wird.

---

25. Separate `clean` -Funktion

Neben dem Artifact Cleanup während `migrate` besitzt das Tool auch einen separaten `clean` -Befehl.

Für einen normalen Testfall:

```
startToscaMigration.bat clean <appID> <testsuite.yaml>
```

Beispiel:

```
startToscaMigration.bat clean PST PST-004_Report.yaml
```

Für ein Template:

```
startToscaMigration.bat clean <appID> <template.yaml> <template.csv>
```

Beispiel:

```
startToscaMigration.bat clean DMP11 template.yaml template.csv
```

Diese Funktion ist von:

```
--cleanup-dry-run
```

```
--cleanup

```

zu unterscheiden.

`clean` dient der inhaltlichen Nachbearbeitung bzw. Sanitization bereits erzeugter AGATE-Dateien.

`--cleanup-dry-run` und `--cleanup` gehören dagegen zum Artifact Audit und zur Verwaltung nicht mehr benötigter Migrationsdateien.

---

26. Manuelle Nachkontrolle

Nach der Migration sollten folgende Punkte geprüft werden:

1. Wurde das erwartete Template YAML erzeugt?
2. Wurde die richtige TCD CSV-Datei erzeugt?
3. Stimmen Template und CSV fachlich zusammen?
4. Sind alle benötigten Reusable-Dateien vorhanden?
5. Sind alle REST-/SOAP-Module vorhanden?
6. Stimmen `command` und Modulpfad überein?
7. Wurden Environment- und Buffer-Mappings korrekt übersetzt?
8. Enthält der Migration Review nicht unterstützte Tosca Module?
9. Gibt es unerwartete Cleanup-Kandidaten?
10. Sind leere oder nicht benötigte Root-Dateien entstanden?

---

27. Leere Root-YAML-Dateien

Bei Template- bzw. TestCase-Design-Migrationen können aktuell zusätzliche Root-YAML-Dateien entstehen, die keinen Inhalt besitzen.

Beispiel:

```
T_AGATE_Demo.yaml       0 Byte
TCD_AGATE_Demo.yaml     0 Byte

```

Wenn der tatsächliche Test unter:

```
template/AGATE_Demo.yaml

```

liegt und die Root-Dateien leer sind, können diese nach einer Prüfung entfernt werden.

Dies ist aktuell Teil der manuellen Nachbearbeitung.

---

28. Vollständiges Beispiel

Schritt 1 – Tosca Export

```
tsu/
├── T_AGATE_Demo.tsu
└── TCD_AGATE_Demo.tsu

```

Schritt 2 – Template zunächst als Dry Run migrieren

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo" --cleanup-dry-run

```

Cleanup-Kandidaten prüfen.

Schritt 3 – Template mit Cleanup migrieren

```
startToscaMigration.bat migrate TOSCATOAGATE "T_AGATE_Demo" --cleanup

```

Schritt 4 – TestCase Design als Dry Run migrieren

```
startToscaMigration.bat migrate TOSCATOAGATE "TCD_AGATE_Demo" --cleanup-dry-run

```

Schritt 5 – TestCase Design migrieren

```
startToscaMigration.bat migrate TOSCATOAGATE "TCD_AGATE_Demo" --cleanup

```

Schritt 6 – Ergebnis prüfen

Erwartete relevante Dateien:

```
migration/data/TOSCATOAGATE/

template/
├── AGATE_Demo.yaml
└── TCD_AGATE_Demo.csv

modules/
└── rest/
    └── v4/
        └── math_request/
            ├── metadata.json
            └── request.json

```

Falls Reusables verwendet wurden:

```
reusable/
└── ...

```

Schritt 7 – Migration Review prüfen

Erwartung:

```
TOSCA MIGRATION REVIEW
============================================================
[SUCCESS] No unprocessed Tosca modules found.

```

Wenn dort Module aufgelistet werden, muss geprüft werden, ob eine manuelle Korrektur erforderlich ist.

Schritt 8 – Nicht benötigte Dateien entfernen

Beispielsweise:

```
0-Byte Root YAML
doppelte bzw. falsch zugeordnete TCD CSV

```

nur nach entsprechender Prüfung entfernen.

---

29. Übergabe an den AGATE Server

Der **AGATE Tosca Migrator ist kein Test Runner**.

Seine Aufgabe endet mit der Erzeugung einer AGATE-kompatiblen Struktur unter:

```
migration/data/<appID>/

```

Nach erfolgreicher Migration, Review und gegebenenfalls manueller Nachbearbeitung kann die erzeugte Applikationsstruktur:

```
migration/data/<appID>

```

in die entsprechende Datenstruktur des:

```
agate-server

```

kopiert bzw. übernommen werden.

Beispiel:

```
agate-tosca-migration
└── migration
    └── data
        └── TOSCATOAGATE
            ├── template
            ├── modules
            └── reusable

                    |
                    | copy
                    v

agate-server
└── ...
    └── data
        └── TOSCATOAGATE
```

```
            ├── template
            ├── modules
            └── reusable
```

Ab diesem Zeitpunkt ist nicht mehr der Migrator für den Test verantwortlich.

Die Ausführung, Validierung und weitere Entwicklung des migrierten Tests erfolgt mit dem **AGATE Server**.

Damit sind die Verantwortlichkeiten klar getrennt:

```
AGATE Tosca Migrator
--------------------
Tosca lesen
Tosca analysieren
Tosca -> AGATE transformieren
Dateien erzeugen
Migration prüfen
Cleanup unterstützen

                ↓

AGATE Server
------------
AGATE Tests laden
Testdaten laden
Testschritte ausführen
REST / SOAP / SQL / CMD / FILE / ... ausführen
Assertions prüfen
Reports erzeugen
```

---

30. Empfohlener Workflow

Für eine neue Migration wird folgende Reihenfolge empfohlen:

```
1. Tosca Template exportieren
2. Tosca TestSheet / TestCase Design exportieren

3. TSU-Dateien unter tsu/ ablegen

4. Template mit --cleanup-dry-run migrieren
5. Cleanup-Ausgabe prüfen
6. Template mit --cleanup migrieren

7. TestCase Design mit --cleanup-dry-run migrieren
8. Cleanup-Ausgabe prüfen
9. TestCase Design mit --cleanup migrieren

10. Migration Review prüfen

11. YAML, CSV, Reusable und API Module prüfen

12. Leere bzw. eindeutig nicht benötigte Artefakte entfernen

13. migration/data/<appID>/ nach agate-server übernehmen

14. Test über AGATE ausführen

15. Fachliches Ergebnis prüfen
```

---

31. Quick Reference

Template Migration

```
startToscaMigration.bat migrate <appID> <template> --cleanup-dry-run
```

danach:

```
startToscaMigration.bat migrate <appID> <template> --cleanup
```

TestCase Design Migration

```
startToscaMigration.bat migrate <appID> <testSheet> --cleanup-dry-run
```

danach:

```
startToscaMigration.bat migrate <appID> <testSheet> --cleanup
```

Testcase Cleanup

```
startToscaMigration.bat clean <appID> <testsuite.yaml>
```

Template Cleanup

```
startToscaMigration.bat clean <appID> <template.yaml> <template.csv>
```

Translation Config überschreiben

```
set AGATE_TOSCA_TRANSLATION_CONFIG=C:\path\customer.yaml
```

Migration-Ergebnis

```
migration/data/<appID>/
```

Testausführung

Nicht im Tosca Migrator.

Das Migrationsergebnis wird in den **agate-server** übernommen und dort mit dem normalen AGATE Test Runner ausgeführt.

---

32. Zusammenfassung

Der AGATE Tosca Migrator ermöglicht die weitgehend automatisierte Überführung bestehender Tosca-Testfälle nach AGATE.

Dabei werden sowohl technische Testschritte als auch TestCase-Design-Daten, Reusable-Bausteine und API-Definitionen berücksichtigt.

Die Migration besteht grundsätzlich aus drei Phasen:

```
Tosca Export
     ↓
```

```
Migration und Review
     ↓
AGATE Artefakte

```

Die erzeugten Artefakte unter:

```
migration/data/<appID>/

```

stellen das Übergabeergebnis des Migrators dar.

Nach einer fachlichen Prüfung werden sie in den **agate-server** übernommen.

Dort beginnt der eigentliche AGATE-Testbetrieb.

Der Migrator selbst bleibt damit bewusst ein eigenständiges Werkzeug für:

> ### Tosca → AGATE Migration

und ist nicht für die spätere Ausführung der migrierten Tests verantwortlich.